

Aula 1: Introdução ao FastAPI e Rotas Básicas

Objetivo da Aula: Configurar o ambiente de desenvolvimento Python, compreender a estrutura básica de uma aplicação web (cliente-servidor) e criar o primeiro endpoint que retorna o inventário atual do laboratório.

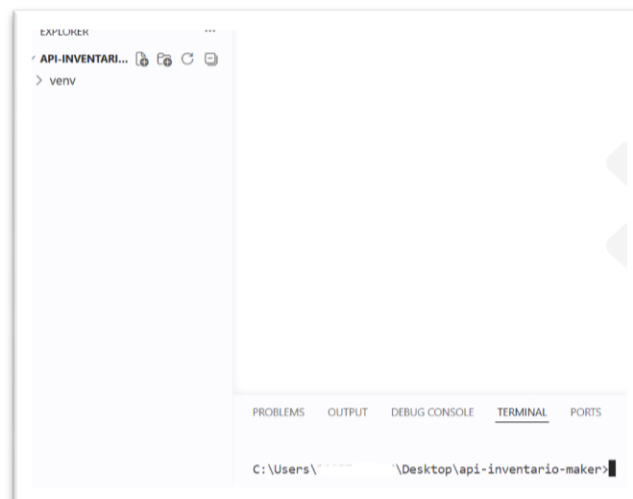
- Introdução e Primeiro Endpoint
- Setup do FastAPI.
- Criação da rota GET /componentes que retorna uma lista estática (em memória) contendo algumas peças do laboratório para testar o protocolo HTTP.
- Exploração do Swagger UI.

1. Preparação do Terreno (Setup do Ambiente)

Antes de escrever qualquer código, é crucial entender a importância do isolamento do projeto.

Comandos no terminal:

- Criar a pasta do projeto: `mkdir api-inventario-maker` e `cd api-inventario-maker`



- Criar o ambiente virtual: `python -m venv venv`

```
C:\Users\...\Desktop\api-inventario-maker>python -m venv venv
```

- Ativar o ambiente: `.\venv\Scripts\activate`

```
C:\Users\          \Desktop\api-inventario-maker>.\venv\Scripts\activate
(venv) C:\Users\          \Desktop\api-inventario-maker>
```

- Instalação das bibliotecas core: `pip install fastapi uvicorn`

```
(venv) C:\Users\          \Desktop\api-inventario-maker>pip install fastapi uvicorn
Collecting fastapi
  Downloading fastapi-0.136.3-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn
  Downloading uvicorn-0.48.0-py3-none-any.whl.metadata (6.7 kB)
Collecting starlette>=0.46.0 (from fastapi)
  Downloading starlette-1.1.0-py3-none-any.whl.metadata (6.3 kB)
Collecting pydantic>=2.9.0 (from fastapi)
  Downloading pydantic-2.13.4-py3-none-any.whl.metadata (109 kB)
Collecting typing-extensions>=4.8.0 (from fastapi)
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
```

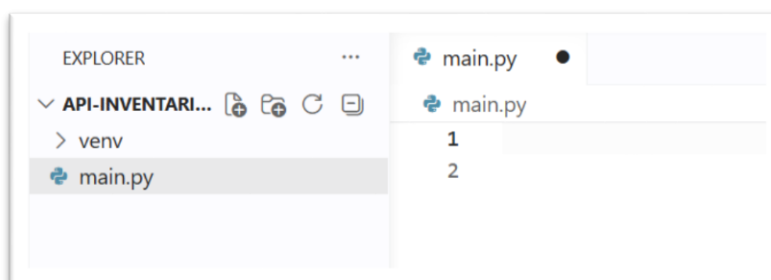
```
Installing collected packages: typing-extensions, idna, h11, colorama, anno
re, click, anyio, uvicorn, starlette, pydantic, fastapi
Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.13
dna-3.16 pydantic-2.13.4 pydantic-core-2.46.4 starlette-1.1.0 typing-extens
```

```
[notice] A new release of pip is available: 25.3 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
(venv) C:\Users\          \Desktop\api-inventario-maker>
```

2. O Código Base (O "Hello World" do Laboratório)

Criar um arquivo chamado `main.py`. A ideia é começar pequeno e ir escalando o código.



```

from fastapi import FastAPI
# Instanciando a aplicação com um título amigável para a documentação
app = FastAPI(title="API Inova Lab - Inventário Maker")
# Nosso "Banco de Dados" em memória para esta primeira fase
estoque_laboratorio = [
    {"id": 1, "nome": "Arduino Sensor Shield", "quantidade": 15, "categoria": "Placas de Expansão"},
    {"id": 2, "nome": "Micro Servo Motor SG90", "quantidade": 42, "categoria": "Atuadores"},
    {"id": 3, "nome": "Esteira em Acrílico", "quantidade": 2, "categoria": "Mecânica"},
    {"id": 4, "nome": "Kit Braço Robótico", "quantidade": 5, "categoria": "Kits"}
]
# Rota de boas-vindas (Raiz)
@app.get("/")
def raiz():
    return {"mensagem": "API do Laboratório Maker operante. Acesse /docs para ver a documentação."}
# Nosso primeiro endpoint real
@app.get("/componentes")
def listar_componentes():
    return estoque_laboratorio

```



```

main.py X
main.py
1  from fastapi import FastAPI
2  # Instanciando a aplicação com um título amigável para a documentação
3  app = FastAPI(title="API Inova Lab - Inventário Maker")
4  # Nosso "Banco de Dados" em memória para esta primeira fase
5  estoque_laboratorio = [
6      {"id": 1, "nome": "Arduino Sensor Shield", "quantidade": 15, "categoria": "Placas de Expansão"},
7      {"id": 2, "nome": "Micro Servo Motor SG90", "quantidade": 42, "categoria": "Atuadores"},
8      {"id": 3, "nome": "Esteira em Acrílico", "quantidade": 2, "categoria": "Mecânica"},
9      {"id": 4, "nome": "Kit Braço Robótico", "quantidade": 5, "categoria": "Kits"}
10 ]
11 # Rota de boas-vindas (Raiz)
12 @app.get("/")
13 def raiz():
14     return {"mensagem": "API do Laboratório Maker operante. Acesse /docs para ver a documentação."}
15 # Nosso primeiro endpoint real
16 @app.get("/componentes")
17 def listar_componentes():
18     return estoque_laboratorio

```

Obs: *Decorators* (@app.get). Os *Decorators* são os "fiscais de trânsito" que dizem para o Python: "Se alguém bater na porta **/componentes** pedindo dados (GET), execute esta função".

3. Colocando no Ar (O Servidor Uvicorn)

Com o código salvo, é hora de rodar a aplicação. Rode o comando do Uvicorn no terminal:

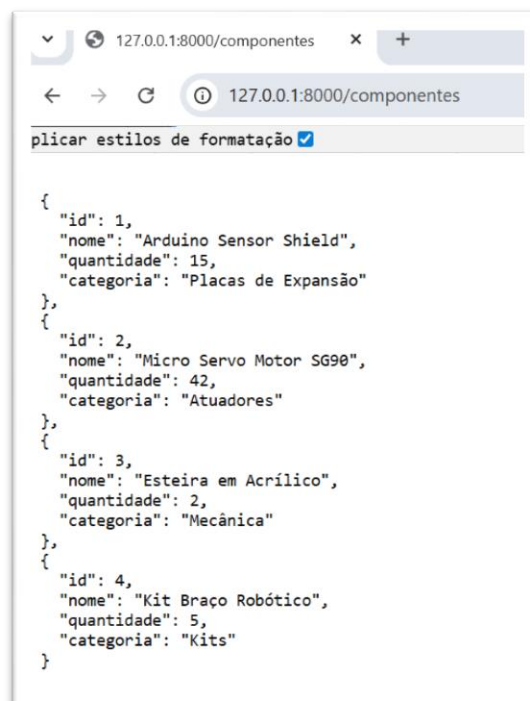
```
uvicorn main:app --reload
```

```
(venv) C:\Users\... \Desktop\api-inventario-maker>uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\19257400808\\Desktop\\api-inventario-maker']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [2980] using StatReload
INFO: Started server process [21140]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

- Anatomia do comando: main é o nome do arquivo .py, app é a variável do FastAPI criada dentro dele, e o --reload é a mágica que reinicia o servidor automaticamente a cada **Ctrl+S** no código.

4. A Mágica da Interface Automática (O Efeito "Uau")

Abrir o navegador e acessar <http://127.0.0.1:8000/componentes>. Será exibido o arquivo JSON cru. É aqui que entra o grande diferencial de usar o FastAPI.



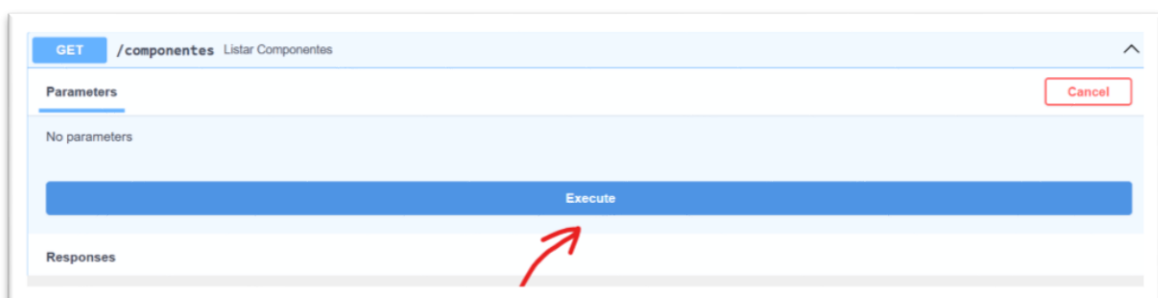
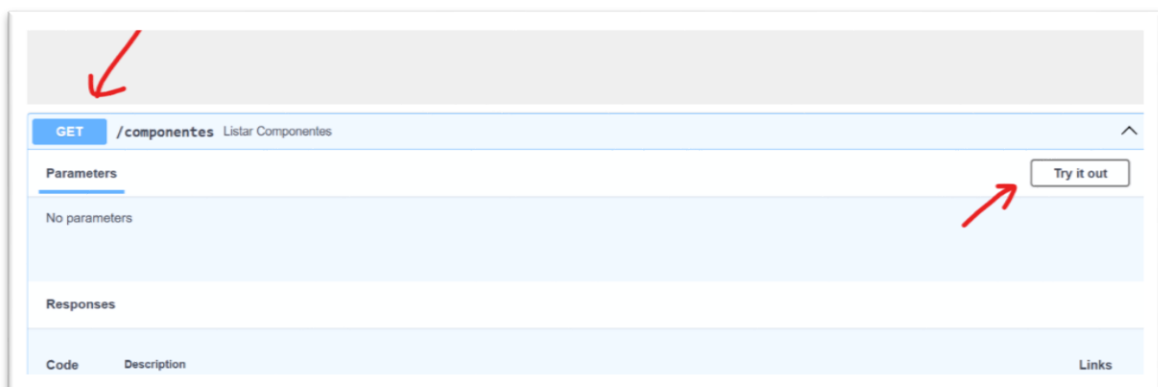
```
{
  "id": 1,
  "nome": "Arduino Sensor Shield",
  "quantidade": 15,
  "categoria": "Placas de Expansão"
},
{
  "id": 2,
  "nome": "Micro Servo Motor SG90",
  "quantidade": 42,
  "categoria": "Atuadores"
},
{
  "id": 3,
  "nome": "Esteira em Acrílico",
  "quantidade": 2,
  "categoria": "Mecânica"
},
{
  "id": 4,
  "nome": "Kit Braço Robótico",
  "quantidade": 5,
  "categoria": "Kits"
}
```

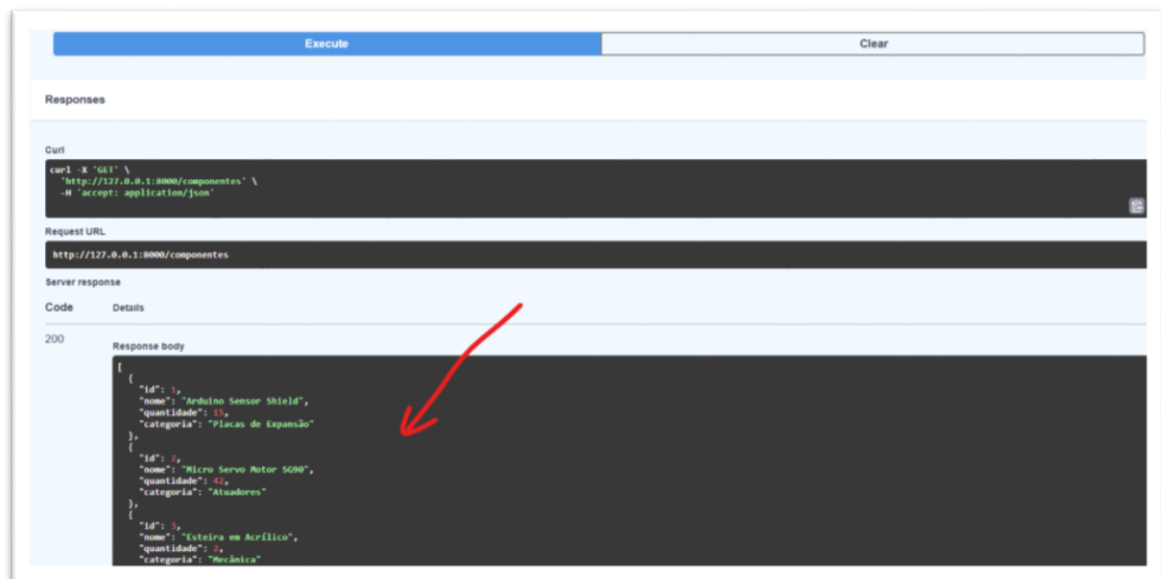
Acessar: <http://127.0.0.1:8000/docs>

- Visualizar a interface do **Swagger UI**.



- Expandir a rota GET /componentes, clicar em "Try it out" e depois em "Execute".



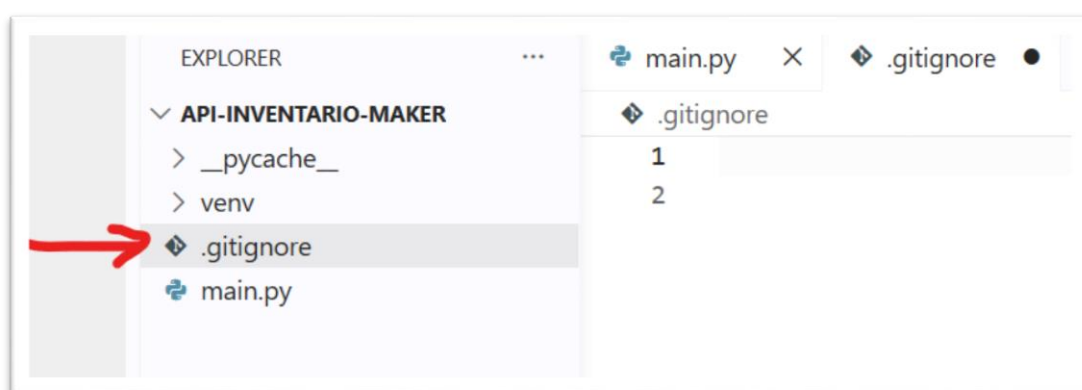


- No mercado de trabalho, essa documentação é o "contrato" que os desenvolvedores *backend* entregam para quem vai construir o *frontend* ou o aplicativo mobile.

5. Criando o .gitignore (A Regra do que NÃO Salvar)

Nunca devemos enviar a pasta do ambiente virtual (venv) para o GitHub, pois ela é muito pesada e dependente do sistema operacional de cada máquina.

- **Passo a passo no terminal:** No terminal (ainda dentro da pasta do projeto `api-inventario-maker`), criar um arquivo chamado `.gitignore`.



- **Conteúdo do arquivo .gitignore:** Abram este arquivo no editor de código e adicionem as seguintes linhas para ignorar o ambiente virtual e os arquivos de cache do Python:

```
# Ambientes Virtuais
```

```
venv/
```

```
env/
```

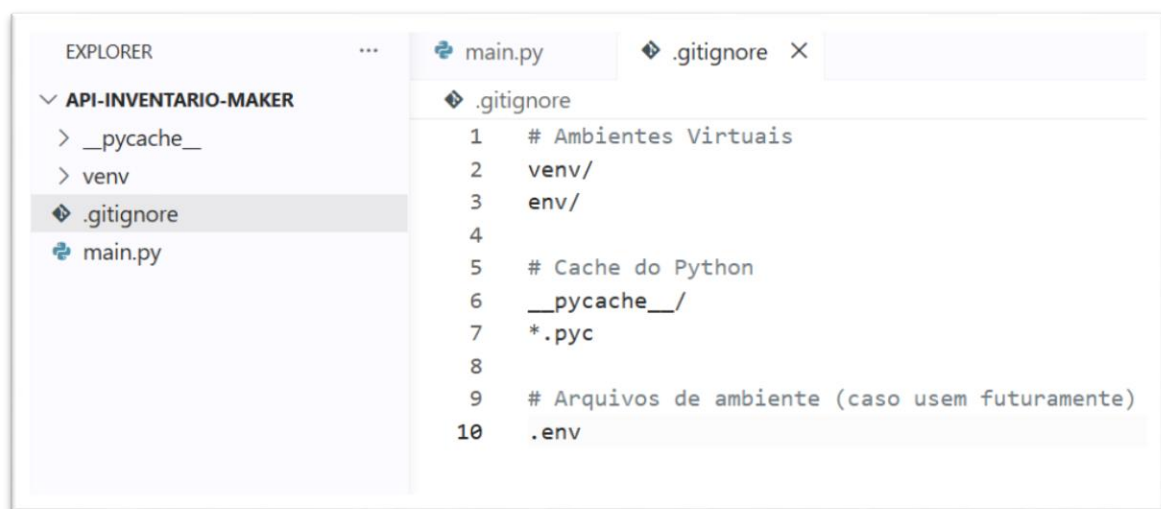
```
# Cache do Python
```

```
__pycache__/
```

```
*.pyc
```

```
# Arquivos de ambiente (caso usem futuramente)
```

```
.env
```



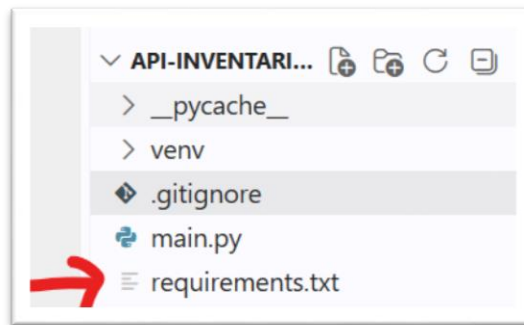
6. Gerando o requirements.txt (A Lista de Ingredientes)

Como o **venv** não vai para o GitHub, quem baixar o projeto precisa saber quais bibliotecas instalar (FastAPI, Uvicorn, etc.).

- **Passo a passo no terminal:** Com o ambiente virtual ativado (o **(venv)** deve estar visível no início da linha do terminal), execute o comando de congelamento do pip:

- `pip freeze > requirements.txt`

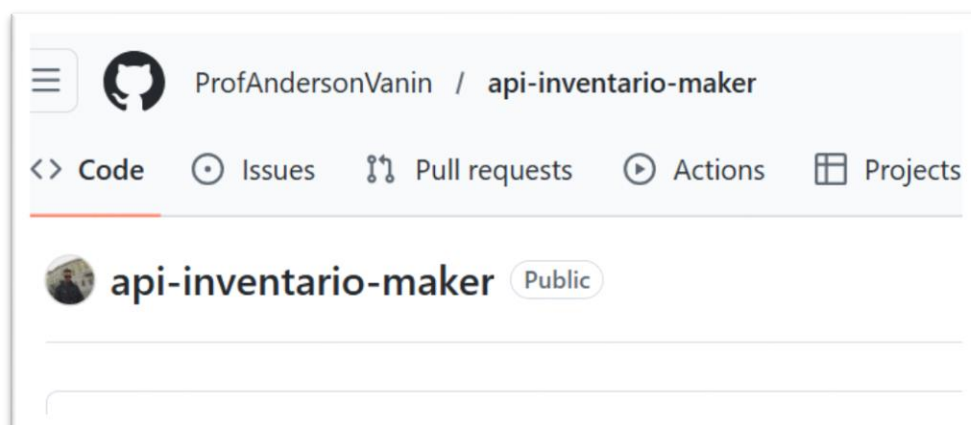
```
(venv) C:\Users\.....\Desktop\api-inventario-maker>pip freeze > requirements.txt
```



- **O Resultado:** Isso criará um arquivo `requirements.txt` na raiz do projeto com as versões exatas de tudo o que foi instalado hoje.

7. O Envio para o GitHub (Passo a Passo)

Criar um repositório vazio em suas contas no GitHub (ex: `api-inventario-maker`).



Passo 1: Inicializar o repositório local

```
git init
```

Isso transforma a pasta comum em um repositório Git.

```
(venv) C:\Users\          \Desktop\api-inventario-maker>git init
Initialized empty Git repository in C:/Users/          /Desktop/api-inventario-maker/.git/
```

Passo 2: Adicionar os arquivos ao palco (staging)

Isso prepara os arquivos (respeitando o que foi ignorado no `.gitignore`). O ponto `.` significa "tudo".


```
git add .
```

```
(venv) C:\Users\          \Desktop\api-inventario-maker>git add .
```

Passo 3: Criar o primeiro "pacote" de alterações (Commit)

Escrevam mensagens claras e semânticas.

```
git commit -m "feat: aula 01 - setup inicial e endpoints em memoria"
```

```
(venv) C:\Users\          \Desktop\api-inventario-maker>git commit -m "feat: aula 01 - setup inicial e endpoints em memoria"
[master (root-commit) f651f04] feat: aula 01 - setup inicial e endpoints em memoria
Committer: ANDERSON SILVA VANIN <          @professores.ibmec.edu.br>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

3 files changed, 42 insertions(+)
create mode 100644 .gitignore
create mode 100644 main.py
create mode 100644 requirements.txt
```

Passo 4: Renomear a branch principal

Por padrão, sistemas mais antigos podem criar a branch como master. O padrão atual é main.

```
git branch -M main
```

```
(venv) C:\Users\19257400808\Desktop\api-inventario-maker>git branch -M main
```

Passo 5: Conectar ao repositório remoto (GitHub)

Copiar a URL do repositório vazio que criaram no GitHub.

```
git remote add origin https://github.com/ProfAndersonVanin/api-inventario-maker.git
```

```
(venv) C:\Users\19257400808\Desktop\api-inventario-maker>git remote add origin https://github.com/ProfAndersonVanin/api-inventario-maker.git
```

Passo 6: Empurrar o código para a nuvem

O comando final para enviar tudo.

```
git push -u origin main
```

```
git push -u origin main
info: please complete authentication in your browser...r>
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.08 KiB | 1.08 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ProfAndersonVanin/api-inventario-maker.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Atualizar a página do repositório no navegador. Devem ser vistos os arquivos `main.py`, `.gitignore` e `requirements.txt` online. Agora o trabalho está oficialmente registrado e visível em seus portfólios.

